

LA CANTINA
Object Design Document
Versione 1.2



Progetto: LA CANTINA	Versione: 1.1
Documento: Object Design Document	Data: 27/12/2025

Coordinatore del progetto:

Nome	Matricola
GRAZIOSO Fabrizio	0512122950

Partecipanti:

Nome	Matricola
LIGUORI Alessandro	0512119887
CICALESE Gabriele	0512116443
GRAZIOSO Fabrizio	0512122950

Scritto da:	GRAZIOSO Fabrizio, CICALESE Gabriele
--------------------	--------------------------------------

Revision History

Data	Versione	Descrizione	Autore
22/12/2025	1.0	Scrittura documento	Grazioso, Cicalese, Liguori
27/12/2025	1.1	Aggiunta dei Package e delle Classi implementate	CICALESE Gabriele
31/12/2025	1.3	Modifica Package e Classi	CICALESE Gabriele

Sommario

1.	Introduzione	4
1.1.	Object design trade-offs	4
1.2.	Interface documentation guidelines	4
1.3.	Definitions, acronyms, and abbreviations	5
1.4.	References	5
2.	Src	5
2.1.	it.unisa.lacantina.model.domain	5
2.2.	It.unisa.lacantina.model.dao	5
2.3.	It.unisa.lacantina.model.service	5
2.4.	It.unisa.lacantina.controller	6
3.	Package	9
4.	Class Interfaces	12
4.1.	Sottosistema UserManagement	12
	Class: UtenteDAO	12
4.2.	Sottosistema CatalogManagement	13
	Class: ProdottoDao	13
4.3.	Sottosistema OrderManagement	17
	Class: RigaOrdineDao	17
	Class: OrdineDao	19
	Class: CarrelloService	21

1. Introduzione

1.1. Object design trade-offs

Durante la definizione del design degli oggetti per il sistema "La Cantina", il team di sviluppo ha dovuto affrontare diverse decisioni architetturali (trade-offs) per bilanciare le prestazioni, la consistenza dei dati e la manutenibilità, in linea con i *Design Goals* definiti nel SDD

- **Buy vs. Build:** Si è optato per un approccio **Build** (sviluppo da zero) utilizzando tecnologie standard Java EE (Servlet/JSP) invece di utilizzare piattaforme e-commerce precostituite. Questa scelta è dettata dalla necessità didattica di avere il controllo completo sul codice e di implementare logiche specifiche per la "filiera corta" e il km0.
- **Memory Space vs. Response Time (Gestione Carrello):** Per la gestione del carrello (Cart), si è scelto di mantenere lo stato interamente in memoria all'interno della HttpSession del server, senza persistenza immediata sul database
 - *Trade-off:* Questa scelta sacrifica l'occupazione di memoria RAM sul server (aumentando il carico per utente attivo) per garantire tempi di risposta (Response Time) immediati durante la navigazione e l'aggiunta di prodotti, evitando continue query di scrittura al DB prima del checkout
- **Data Access Strategy (JDBC vs ORM):** È stato scelto l'utilizzo di JDBC nativo con PreparedStatement invece di framework ORM (come Hibernate)
 - *Trade-off:* Sebbene questo comporti una maggiore verbosità nel codice dei DAO, garantisce un controllo granulare sulle query SQL, ottimizzando le performance e la sicurezza (prevenzione SQL Injection), riducendo l'overhead di astrazione.

1.2. Interface documentation guidelines

Per garantire la coerenza tra i moduli sviluppati dai diversi membri del team, sono state stabilite le seguenti convenzioni di codifica e documentazione delle interfacce:

- **Nomenclatura Classi:** Le classi devono essere nominate utilizzando sostantivi al singolare in stile PascalCase (es. Prodotto, Ordine), mantenendo la lingua italiana per coerenza con il dominio
- **Nomenclatura Metodi:** I metodi devono utilizzare verbi o frasi verbali in stile camelCase.
 - I metodi dei DAO devono seguire il suffisso standard DAO (es. UserDao, ProdottoDAO, OrdineDAO).
 - I metodi dei Controller devono riflettere l'azione (es. processCheckout, login).
- **Gestione Errori:** Lo stato di errore non deve essere restituito tramite valori interi, ma sollevando eccezioni specifiche. Le eccezioni SQL (SQLException) devono essere catturate e rilanciate come eccezioni applicative (es. RuntimeException o custom DAOException) per non esporre dettagli del database al livello superiore.
- **Collezioni:** I metodi che restituiscono collezioni (es. List<Prodotto>) non devono mai restituire null, ma una lista vuota (isEmpty()) in assenza di risultati, per evitare NullPointerException.

- **Visibilità:** Tutti gli attributi delle classi Entity devono essere private. L'accesso deve avvenire esclusivamente tramite metodi pubblici get e set.

1.3. *Definitions, acronyms, and abbreviations*

- **DAO (Data Access Object):** Pattern utilizzato per astrarre e incapsulare l'accesso ai dati persistenti.
- **MVC (Model-View-Controller):** Pattern architetturale su cui si basa l'intero sistema.
- **Entity:** Oggetto che rappresenta un dato persistente nel dominio (es. Utente, Prodotto)
- **Session Bean:** Oggetto mantenuto nella sessione utente (es. Carrello)
- **SDD:** System Design Document.
- **RAD:** Requirements Analysis Document.

1.4. *References*

- *Requirements Analysis Document (RAD) v2.1* - La Cantina.
- *System Design Document (SDD) v2.0* - La Cantina.

2. **Src**

All'interno delle cartelle del progetto è presente una sottocartella Database contenente il codice sql mentre un'altra sottocartella IMG contiene tutti i file multimediali utilizzati. Inoltre, il codice sorgente è organizzato in package Java che riflettono la decomposizione in sottosistemi definita nel SDD e l'architettura MVC.

2.1. *it.unisa.lacantina.model.domain*

Contiene tutti gli oggetti del dominio applicativo descritte sotto forma di classi, tali oggetti fanno riferimento alle entità presenti nel database e memorizzate all'interno della sessione utente come per la classe "Carrello". Tali classi **rappresentano i dati del dominio** e non contengono logica di business complessa ma solo attributi e metodi di accesso.

- **Classi implementate:** Utente, Prodotto, Ordine, RigaOrdine, Carrello, Fornitore.
- **Dipendenze:** Nessuna dipendenza verso altri package del sistema.

2.2. *It.unisa.lacantina.model.dao*

Contiene le classi che implementano il pattern **DAO**. Gestisce la comunicazione diretta con il DBMS MySQL tramite JDBC. Questo package corrisponde al livello "Persistent data management".

- **Classi implementate:** UtenteDAO, ProdottoDAO, OrdineDAO, RigaOrdineDAO, FornitoreDao.
- **Dipendenze:** it.unisa.lacantina.model (utilizza le entity per input/output), java.sql, java.util.

2.3. *It.unisa.lacantina.model.service*

Contiene la classe CarrelloService che implementa i metodi per il comportamento del carrello salvato come variabile di sessione.

Classi implementate: CarrelloService.

- **Dipendenze:** it.unisa.lacantina.model, java.util.

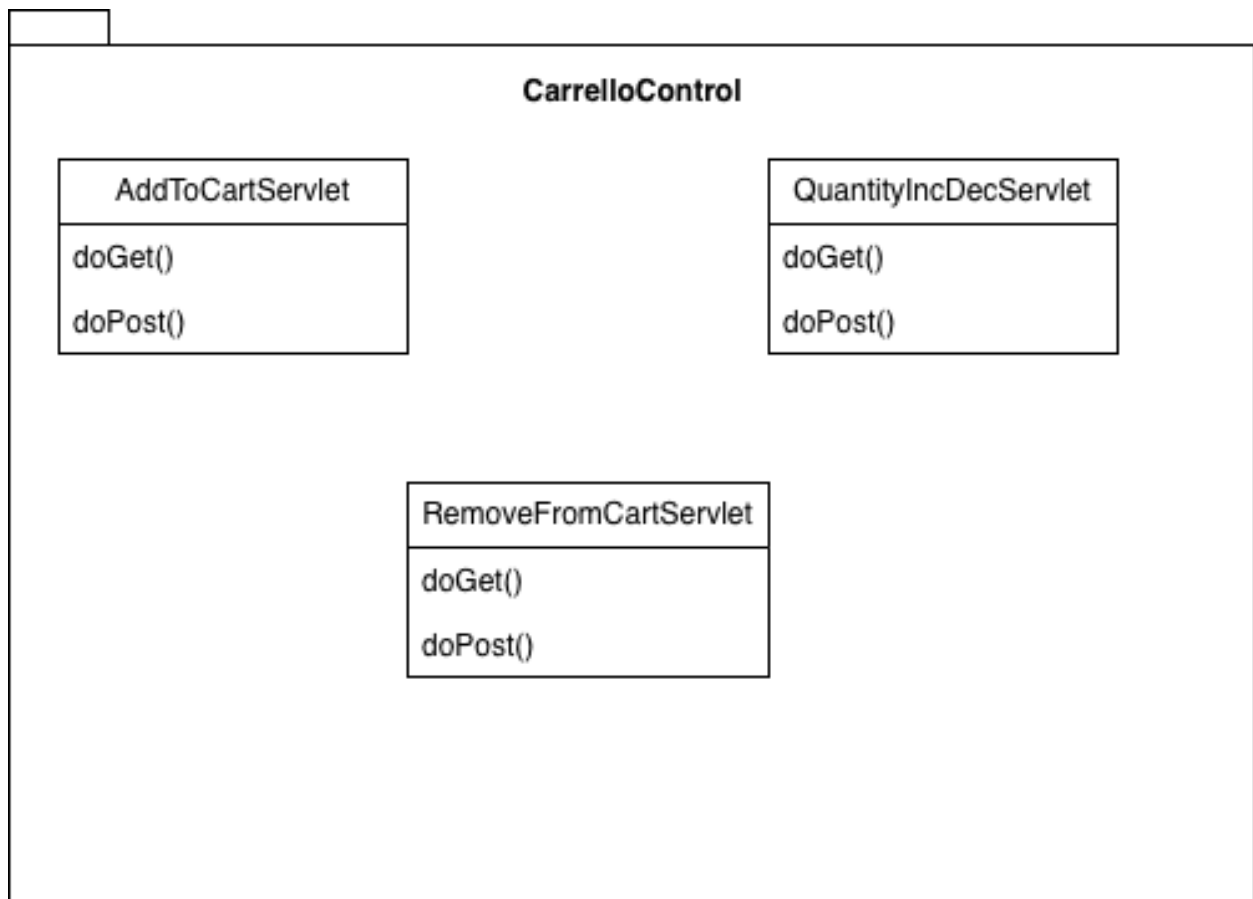
2.4 *It.unisa.lacantina.controller*

Contiene le **Servlet** che fungono da Controller. Questo package gestisce la logica applicativa, riceve gli input dalla View (JSP), interagisce con i DAO, e determina la pagina di risposta con l'esito dell'operazione eseguita dall'utente.

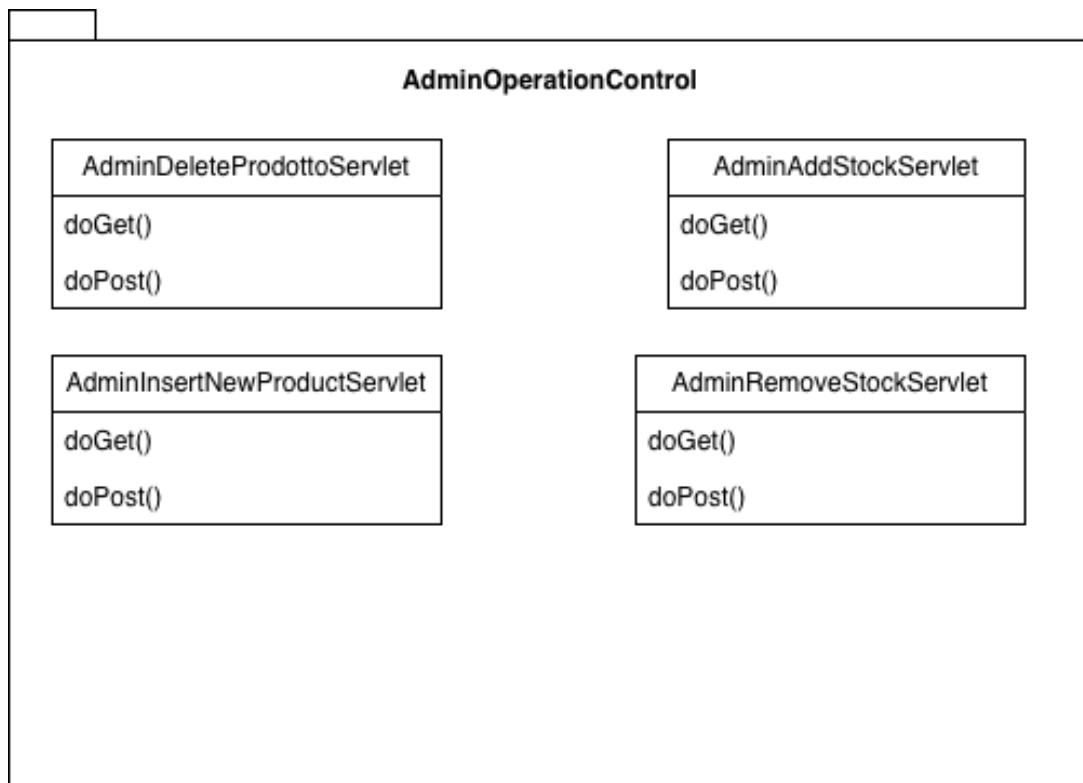
- **Classi implementate:** AddToCartServlet, AdminAddStockServlet, AdminDeleteProdottoServlet, AdminInsertNewProductServlet, AdminRemoveStockServlet, CheckOutServlet, LoginServlet, LogoutServlet, ModifyOrderDataServlet, OrderNowServlet, QuantityIncDecServlet, RegisterServlet, RemoveFromCartServlet
- **Dipendenze:** it.unisa.lacantina.model, it.unisa.lacantina.model.dao, it.unisa.lacantina.util

Il package del controller è a sua volta suddiviso in ulteriori package tali da raggruppare le varie servlet per le funzionalità espletate

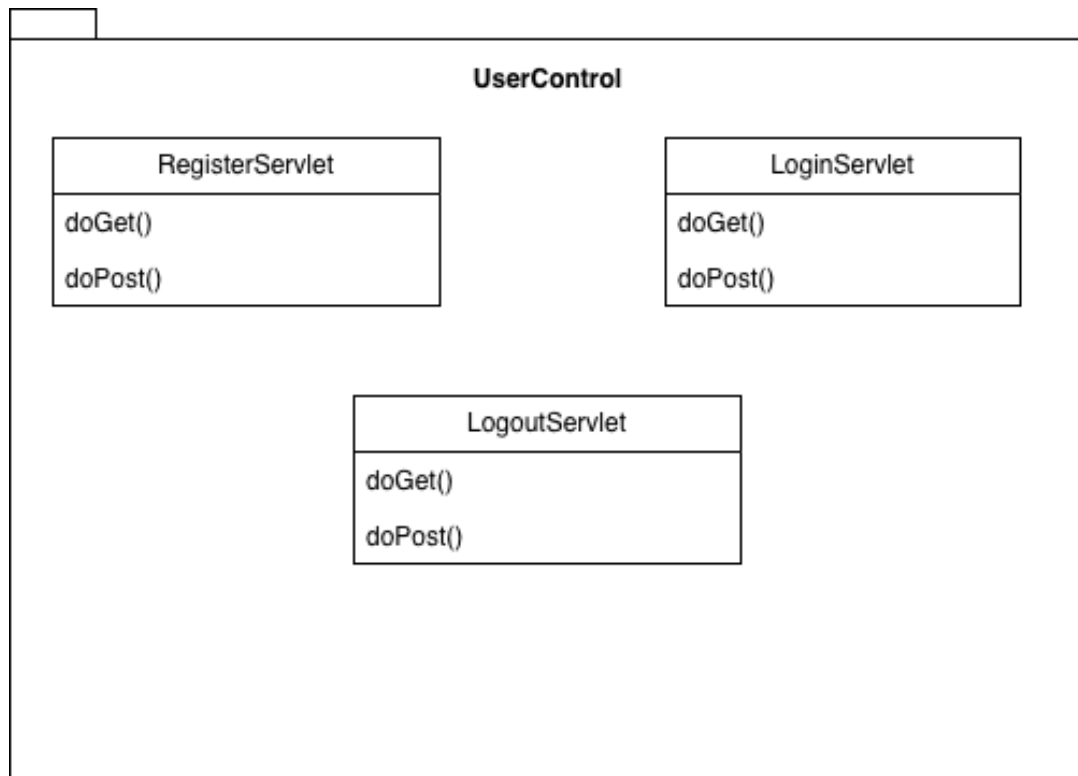
Controller.CarrelloControl



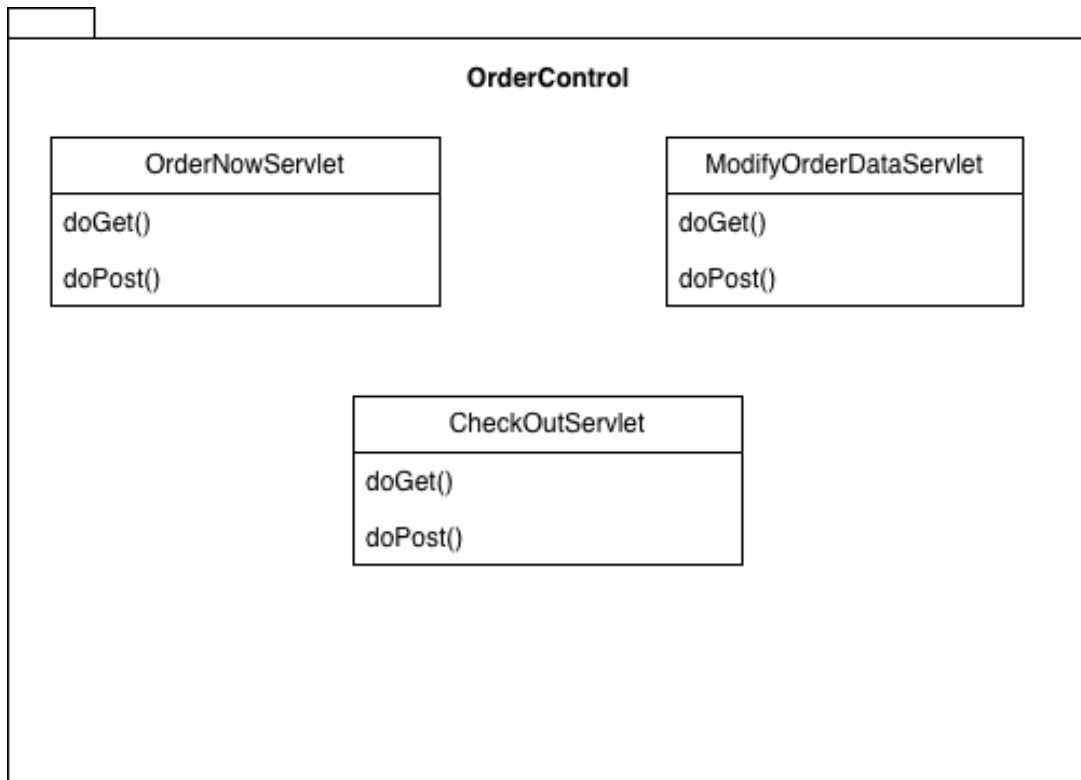
Controller.AdminOperationControl



Controller.UserControl



Controller.OrderControl



2.5 *it.unisa.lacantina.util*

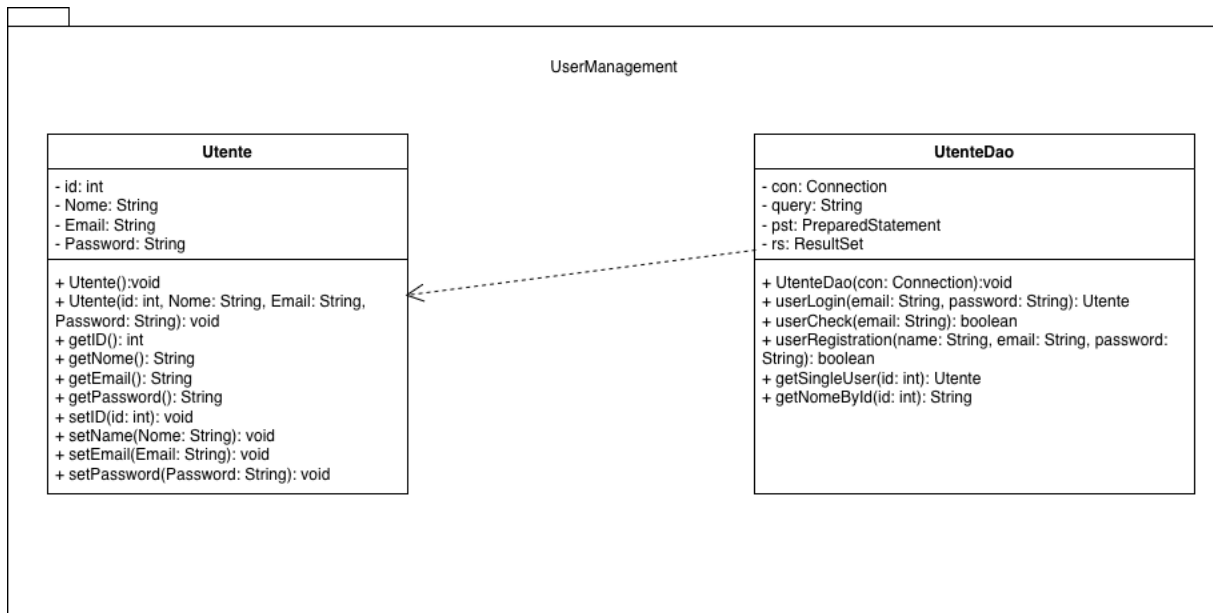
Contiene unicamente la classe Java *ConnectToDB* che sfrutta *JDBC* per effettuare la connessione al database impostando i parametri di root.

- **Classi implementate:** *ConnectToDB*
- **Dipendenze:** *java.sql*

3. Package

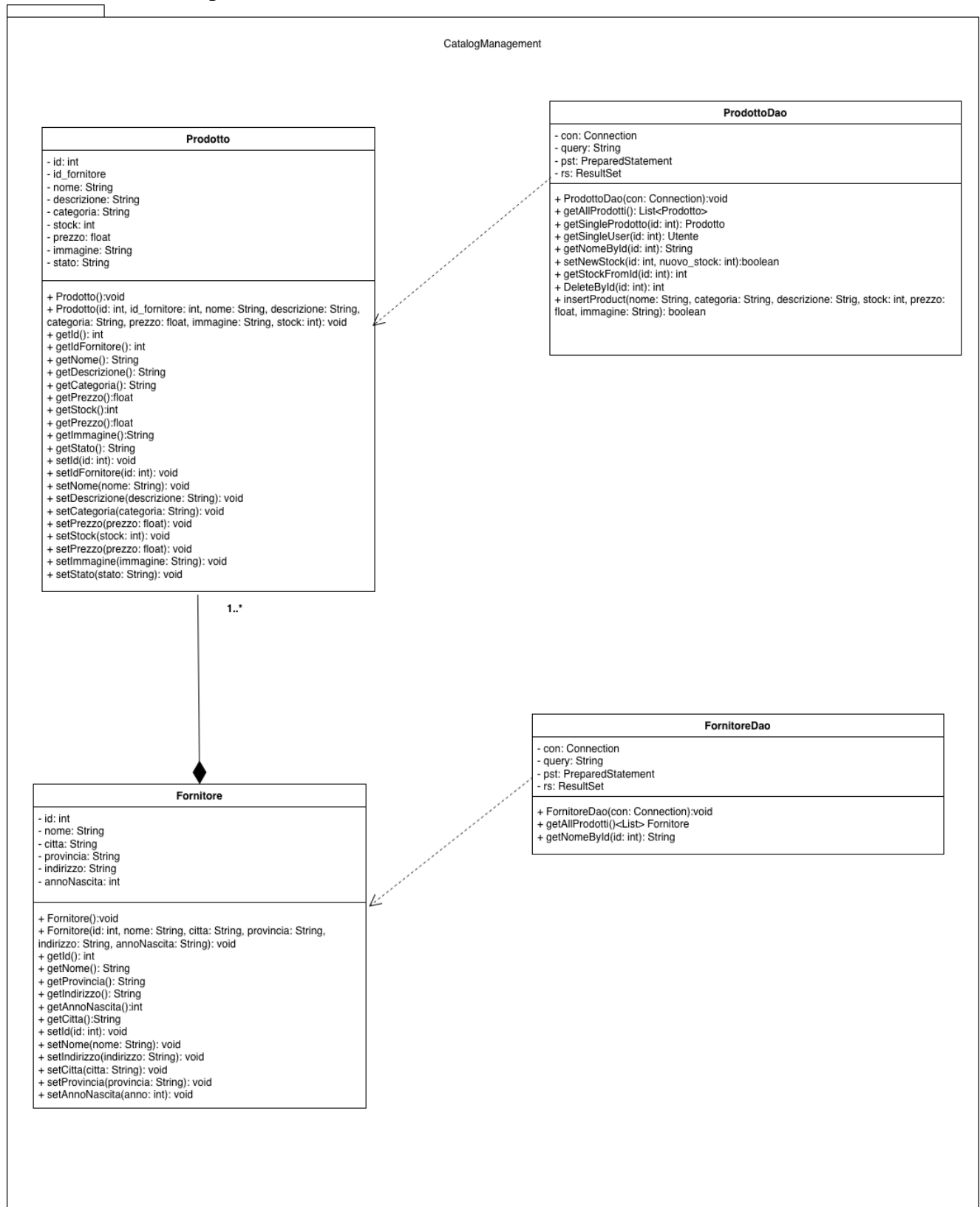
3.1 UserManagement

Questo package contiene le classi degli oggetti del dominio applicativo con le relative classi DAO per la manipolazione dei rispettivi dati sul Database **in merito alla gestione dell'utente**.



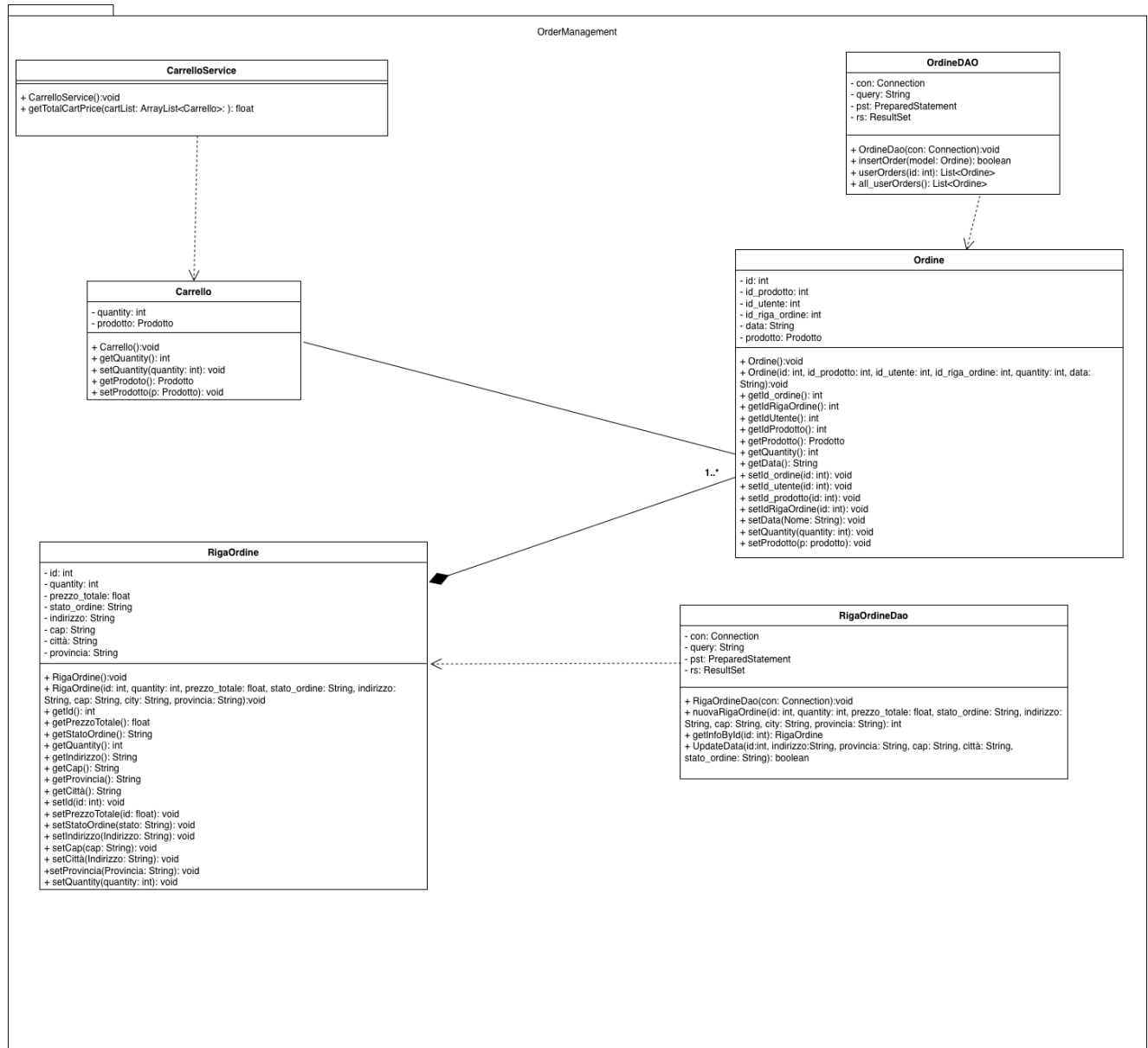
3.2 CatalogManagement

Questo package contiene le classi degli oggetti del dominio applicativo con le relative classi DAO per la manipolazione dei rispettivi dati sul Database **in merito alla visualizzazione dei prodotti all'interno dello shop del sito.**



3.3 OrderManagement

Questo package contiene le classi degli oggetti del dominio applicativo con le relative classi DAO per la manipolazione dei rispettivi dati sul Database **in merito alla gestione degli acquisiti dei clienti con il ServiceCarrello** per la gestione del carrello.



4. Class Interfaces

Questa sezione descrive le interfacce pubbliche delle classi chiave per i sottosistemi da implementare. **Per tutte le classi vale la regola che la connessione al database sia effettuata e che quindi la variabile di sessione con non sia null, ossia, pre: self.con<null.**

4.1. Sottosistema UserManagement

Class: UtenteDAO

UtenteDao	
Descrizione Classe	Questa classe si interfaccia con il DB, permettendo di gestire le operazioni relative all'account utente
Pre-condizioni	Context UtenteDao::userRegistration(nome:String, email: String, password: String) Pre: nome<null and email< null and Utente.allInstances()->forAll(u u.email<email) and password <null Context UtenteDao::userLogin(email: String, password: String) Pre: email< null and password <null Context UtenteDao::getNomeById(id: Integer) Pre: id>0 and Utente.allInstances()->exists(u u.id = id) Context UtenteDao::getSingleUser(id: Integer) Pre: id>0 and Utente.allInstances()->exists(u u.id = id) Context UtenteDao::UserCheck (email: String) Pre: email<null
Post-condizioni	Context UtenteDao::userRegistration(nome:String, email: String, password: String) Post: Utente.allInstances()->exists(u u.nome = nome and u.email = email and u.password = password) Context UtenteDao::userLogin(email: String, password: String) Post: result <null implies Utente.allInstances()->exists(u u.email = email and u.password = password) and (result = null) implies Utente.allInstances()->forAll(u.email<email or u.password<password) Context UtenteDao::getNomeById(id: Integer) Post: result = Utente.allInstances()->any(u u.id = id).nome

	Context UtenteDao::getSingleUser(id: Integer) Post: (result $\neq$ null implies Utente.allInstances()->exists(u u.id = id and u = result)) and (result = null implies Utente.allInstances()->forall(u u.id $\neq$ id)) Context UtenteDao::UserCheck (email: String) Post: (result=true implies Utente.allInstances()->exists(u u.email =email)) and (result = false implies Utente.allInstances()->forall(u u.email $\neq$ email))
Invarianti	Context Utente inv: id>0 inv: email $\neq$ null inv:password $\neq$ null

4.2. Sottosistema CatalogManagement

Class: ProdottoDao

ProdottoDao	
Descrizione Classe	Questa classe si interfaccia con il DB, permettendo di gestire i prodotti registrati sul DB.
Pre-condizioni	Context ProdottoDao::getAllProdotti() Pre: true Context ProdottoDao::getSingleProdotto(id: Integer) Pre: id > 0 and Prodotto.allInstances()->exists(p p.id = id) Context ProdottoDao::setNewStock(id: Integer, stock: Integer) Pre: stock> 0 and id>0 and Prodotto.allInstances()->exists(p p.id = id) Context ProdottoDao::getStockFromId(id: Integer) Pre: id>0 and Prodotto.allInstances()->exists(p p.id = id) Context ProdottoDao::DeleteById(id: Integer) Pre: id>0 and Prodotto.allInstances()->exists(p p.id = id & p.stato = 'attivo')

	Context ProdottoDao:InsertProduct(nome: String, id_fornitore: Integer, categoria: String, descrizione: String, immagine: String, stock: Integer, prezzo: Real) Pre: nome $\diamond$ null and categoria $\diamond$ null and descrizione $\diamond$ null and immagine $\diamond$ null and stock > 0 and prezzo > 0 and id_fornitore > 0 and Fornitore.allInstances()->exists(f f.id = id_fornitore)
Post-condizioni	Context ProdottoDao::getAllProdotti() Post: result->forAll(p p.id > 0 and p.id_fornitore > 0 and p.nome $\diamond$ null and p.categoria $\diamond$ null and p.descrizione $\diamond$ null and p.immagine $\diamond$ null and p.prezzo > 0 and p.stock > 0 and p.stato $\diamond$ null) Context ProdottoDao::getSingleProdotto(id: Integer) Post: (result $\diamond$ null implies result.id > 0 and result.id_fornitore > 0 result.nome $\diamond$ null and result.categoria $\diamond$ null and result.descrizione $\diamond$ null and result.immagine $\diamond$ null and result.prezzo > 0 and result.stock > 0 and result.stato $\diamond$ null Context ProdottoDao::setNewStock(id: Integer, stock: Integer) Post: (result = true implies Prodotto.allInstances()->exists(p p.id = id and p.stock = newStock and p.stock >= 0))

	and (result = false implies Prodotto.allInstances()->forAll(p p.id <> id or p.stock >= 0)) Context ProdottoDao::getStockFromId(id: Integer) Post: result >= 0 and (Prodotto.allInstances()->exists(p p.id = id) implies result = Prodotto.allInstances()->any (p p.id = id).stock) Context ProdottoDao::DeleteById(id: Integer) Post: (result = true implies Prodotto.allInstances()->forAll(p p.id = id and p.stato = 'inattivo')) Context ProdottoDao::insertProduct(nome: String, id_fornitore: Integer, categoria: String, descrizione: String, immagine: String, stock: Integer, prezzo: Real) Post: result = true implies Prodotto.allInstances()->exists (p p.nome = nome and p.id_fornitore = id_fornitore and p.categoria = categoria and p.descrizione = descrizione and p.immagine = immagine and p.stock = stock and p.prezzo = prezzo and p.stato = 'attivo')
Invarianti	Context Prodotto inv: id > 0 inv: nome <> null inv: categoria <> null inv: descrizione <> null inv: immagine <> null inv: prezzo > 0 inv: stock > 0 inv: stato <> null

Class: FornitoreDao

FornitoreDao	
Descrizione Classe	Questa classe si interfaccia con il DB, permettendo di gestire i fornitori registrati sul DB.
Pre-condizioni	Context FornitoreDao::getAllFornitori() Pre: true Context FornitoreDao::getNomeById(id: Integer) Pre: id > 0 and Fornitore.allInstances()->exists (f f.id = id)
Post-condizioni	Context FornitoreDao::getAllProdotti() Post: result->forAll(p p.id > 0 and p.nome <> null and p.anno_nascita > 0 and p.citta <> null and p.provincia <> null and p.indirizzo <> null) Context FornitoreDao::getNomeById (id: Integer) Post: (result <> null implies Prodotto.allInstances()->exists(p p.id = id and p.nome = result)
Invarianti	Context Fornitore inv: id > 0 inv: nome <> null inv: citta <> null inv: provincia <> null inv: indirizzo <> null inv: anno_nascita > 0

4.3. Sottosistema OrderManagement

Class: RigaOrdineDao

RigaOrdineDao	
Descrizione Classe	Questa classe si interfaccia con il DB, permettendo di inserire una nuova riga ordine, ossia, un nuovo contenitore logico di acquisto che raggruppa tutti gli ordini effettuati da un utente in una singola operazione di acquisto.
Pre-condizioni	Context:RigaOrdineDao::nuovaRigaOrdine(indirizzo: String, cap: String, provincia: String, prezzo_totale: Real, numero_ordini: Integer, String citta) Pre: indirizzo $\neq$ null and cap$\neq$null and citta$\neq$null and provincia$\neq$null and prezzo_totale>0 and numero_ordini>0 Context:RigaOrdineDao::getInfoById(id: Integer) Pre: id>0 Context:RigaOrdineDao::UpdateData(id: Integer) Pre: id > 0 and RigaOrdine.allInstances()->exists(r r.id = id) and indirizzo$\neq$null and provincia$\neq$null and cap$\neq$null and citta$\neq$null and stato_ordine$\neq$null
Post-condizioni	Context:RigaOrdineDao::nuovaRigaOrdine(indirizzo: String, cap: String, provincia: String, prezzo_totale: Real, numero_ordini: Integer, String citta) Post: result > 0 implies RigaOrdine.allInstances()->exists(r r.id = result and r.indirizzo = indirizzo and r.cap = cap and r.citta = citta and r.provincia = provincia and r.prezzo_totale = prezzo_totale and r.numero_ordini = numero_ordini and r.stato_ordine = “attesa di conferma”

	Context: RigaOrdineDao::getInfoById(id: Integer) Post: (result $\neq$ null implies <pre> RigaOrdine.allInstances()->exists(r r.id = id and result.id = r.id and result.numero_ordini = r.numero_ordini and result.prezzo_totale = r.prezzo_totale and result.stato_ordine = r.stato_ordine and result.indirizzo = r.indirizzo and result.cap = r.cap and result.citta = r.citta and result.provincia = r.provincia)) and (result = null implies RigaOrdine.allInstances()->forall(r r.id $\neq$ id)) </pre> Context: RigaOrdineDao::UpdateData(id: Integer) Post: result = true implies <pre> RigaOrdine.allInstances()->exists(r r.id = id and r.indirizzo = indirizzo and r.provincia = provincia and r.cap = cap and r.citta = citta and r.stato_ordine = stato_ordine) </pre>
Invarianti	Context RigaOrdine <pre> inv: indirizzo $\neq$ null inv: cap $\neq$ null inv: citta $\neq$ null inv: provincia $\neq$ null inv: stato_ordine $\neq$ null inv: prezzo_totale $\neq$ null inv: prezzo_totale > 0.0 inv: quantity > 0 inv: id > 0 </pre>

Class: OrdineDao

OrdineDao	
Descrizione Classe	Questa classe si interfaccia con il DB, permettendo di effettuare acquisti sia istantanei che da carrello e registrarli sul database.
Pre-condizioni	Context:OrdineDao::insertOrder(model:Ordine) Pre: model $\neq$ null and model.idProdotto > 0 and Prodotto.allInstances()->exists(p p.id = model.idProdotto) and model.idUtente > 0 and model.quantity > 0 and model.data $\neq$ null and model.idRigaOrdine > 0 Context:OrdineDao::userOrders(id: Integer) Pre: id>0 and Utente.allInstances()->exists(u u.id = id) Context:OrdineDao::all_userOrders() Pre: true
Post-condizioni	Context:OrdineDao::insertOrder(model:Ordine) Post: result = true implies Ordine.allInstances()->exists(o o.idProdotto = model.idProdotto and o.idUtente = model.idUtente and o.quantity = model.quantity and o.data = model.data and o.idRigaOrdine = model.idRigaOrdine and o.prodotto = model.prodotto) Context:OrdineDao::userOrders(id: Integer) Post: post: result->forAll(o o.idUtente = id and o.id > 0 and o.quantity > 0 and o.data $\neq$ null and o.idRigaOrdine > 0 and o.prodotto $\neq$ null and o.prodotto.id > 0 and o.prodotto.nome $\neq$ null and

	<pre> o.prodotto.categoria <> null and o.prodotto.descrizione <> null and o.prodotto.immagine <> null and o.prodotto.prezzo >= 0 and o.prodotto.stock >= 0 and o.prodotto.stato <> null) Context:OrdineDao::all_userOrders() Post: context OrdineDao::all_userOrders() post: result->forAll(o o.id > 0 and o.idUtente > 0 and o.idRigaOrdine > 0 and o.quantity > 0 and o.data <> null and o.prodotto <> null and o.prodotto.id > 0 and o.prodotto.nome <> null and o.prodotto.categoria <> null and o.prodotto.descrizione <> null and o.prodotto.immagine <> null and o.prodotto.prezzo >= 0 and o.prodotto.stock >= 0 and o.prodotto.stato <> null and o.utente <> null and o.utente.id > 0 and o.utente.nome <> null and o.utente.email <> null and o.utente.password <> null) </pre>
Invarianti	<pre> context Ordine inv: id > 0 inv: id_prodotto > 0 inv: id_utente > 0 inv: quantity > 0 inv: data <> null inv: id_riga_ordine > 0 inv: prodotto <> null inv: prodotto.id = prodotto.id </pre>

Class: CarrelloService

CarrelloService	
Descrizione Classe	La classe CarrelloService gestisce la logica di manipolazione del carrello degli utenti. Fornisce il metodo per calcolare il prezzo totale dei prodotti aggiunti a carrello. I dati sono temporanei e non persistenti , memorizzati nella sessione dell'utente durante la navigazione.
Pre-condizioni	Context CarrelloService::getTotalCartPrice(cartList: Sequence (Carrello)) Pre: cartList->forAll(c c.prodotto <> null and c.prodotto.prezzo >= 0.0 and c.quantity > 0)
Post-condizioni	Context CarrelloService::getTotalCartPrice(cartList: Sequence (Carrello)) Post: result = cartList->iterate(c; sum: Real = 0.0 sum + c.prodotto.prezzo * c.quantity)
Invarianti	context Carrello inv: prodotto <> null inv: prodotto.id > 0 inv: quantity > 0 inv: prodotto.prezzo > 0.0